

Priyanshu Dewangan – CMIT Fellow

Assignment -2 : Document Chunking Strategy

1. The 3 Chunking Strategies Used

I replaced the single hardcoded layout-based chunker with three selectable strategies, depending on document structure and downstream RAG requirements:

1.1 Semantic Chunking

- How it works: It reads the sentences in your document, converts them to embedding vectors (using the BAAI/bge-m3 model), and evaluates the distance between them. A new chunk is created whenever there is a significant shift in topic (meaning).
- *Code Implementation:*

```
□self.embeddings = HuggingFaceEmbeddings(model_name=model)
self.chunker = SemanticChunker(self.embeddings)
□
```

1.2 Hybrid Chunking

- How it works: Uses Docling's native layout chunker. It respects visual and structural boundaries (like tables, sections, and paragraphs) from the document conversion, ensuring layout sections are not broken up, while bounding the total size using a tokenizer.
- *Code Implementation:*

```
□self.tokenizer = AutoTokenizer.from_pretrained(model)
self.chunker = HybridChunker(tokenizer=self.tokenizer,
max_tokens=chunk_size)
□
```

1.3 Recursive Chunking

- How it works: Splits text sequentially using a prioritized list of characters (paragraphs, then sentences, then words) until the text fragments fit within a target size.

- *Code Implementation:*

```
□self.chunker =  
RecursiveCharacterTextSplitter(chunk_size=chunk_size,  
chunk_overlap=chunk_overlap)  
□
```

2. How Max Tokens / Chunk Size is Handled

We consolidated the token limit control under the `--chunk-size` CLI parameter (default is 1024). The strategies handle this limit in different ways:

- Hybrid Chunker: It is passed as `max_tokens`. Docling uses the tokenizer to ensure no single layout chunk exceeds this count.
- Recursive Chunker: It is passed as `chunk_size` (in characters).
- Semantic Chunker: Because it splits dynamically based on semantic changes (topic shifts) between sentences, it doesn't enforce a hard token limit. Instead, it groups sentences together as long as they talk about the same subject.

3. How Overlap is Handled

To make sure context isn't lost between adjacent chunks, we handle overlaps via the `--overlap` parameter (default is 200 characters):

- Recursive Strategy: Fully supports overlap. A sliding window of 200 characters from the end of the previous chunk is prepended to the start of the next chunk. This prevents keyword splits at chunk boundaries.
- Semantic Strategy: Standard sliding-window overlap is not used here because the splitter determines natural boundaries between sentences. The transition from one topic to another is preserved by retaining the heading context.
- Hybrid Strategy: Docling's HybridChunker operates on structural layout blocks (like table cells or complete lists) and does not use character-level overlaps, preventing structural elements from becoming duplicated or fragmented.